

**VISOKA TEHNIČKA ŠKOLA
STRUKOVNIH STUDIJA
SUBOTICA**

**PROJEKAT
iz predmeta Integracija informacionih sistema
LycoWeather - Vremenska Stanica**

KANDIDAT(I)

Stepan Turitsin

David Katrinka

MENTOR

prof. dr Sanja Maravić Čisar

SUBOTICA, 2026.

Sadržaj

1. Opis projekta	2
2. Realizacija Zadatka	3
Tehnologije i biblioteke	3
Struktura projekta	5
Backend	5
Frontend	6
3. Struktura baze podataka	8
4. Opis funkcionalnosti	9
LiveData widget	9
HistoryTable	10
WeatherChart	11
MeteoData	12
5. Korišćena Literatura	13
Web izvori	13

1. Opis projekta

Projekat "LycoWeather" predstavlja rešenje za nadgledanje i praćanje temperature i vlažnosti vazduha, vlažnosti zemljišta i nivoa svetlosti. Same podatke o vremenu sakuplja mikrokontroler ESP-32 sa senzorima za temperaturu i vlažnost vazduha (DHT22), senzor za vlažnost vazduha i senzor za svetlo (foto-otpornik).

Nakon što mikrokontroler prikupi sirove podatke, on ih obrađuje i konvertuje u čitljive vrednosti, izražene u stepenima Celzijusa za temperaturu i u procentima za vlažnost vazduha, vlažnost zemljišta i nivo svetlosti. Tako formatirani podaci se zatim šalju na backend u JSON formatu, a istovremeno se prikazuju i u Serial Monitoru radi praćenja u realnom vremenu.

Backend sistem prima podatke sa mikrokontrolera i čuva ih u bazi podataka, pružajući pritom različite načine za pristup tim podacima. Korisnik može dobiti podatke u realnom vremenu, što prikazuje trenutno stanje senzora mikrokontrolera, pregledati istorijske podatke uz podršku za paginaciju, ili filtrirati podatke po vremenskom periodu u rasponu od 10 minuta do 24 sata. Pored lokalno prikupljenih podataka, backend integriše i informacije sa Open-Meteo API-ja, omogućavajući poređenje i analizu odstupanja između sopstvenih merenja i zvaničnih meteoroloških podataka.

Frontend aplikacija dobija podatke od backend sistema i prikazuje ih korisniku na više načina. Podaci u realnom vremenu prikazuju se u malom widget-u, dok se istorija podataka može pregledati u tabelarnom formatu sa paginacijom. Podaci po vremenskom periodu vizuelno se prikazuju putem grafikona, što omogućava praćenje promena tokom vremena. Pored podataka sa sopstvenih senzora, frontend prikazuje i podatke sa Open-Meteo API-ja za period od pet dana, čime je omogućeno poređenje sa lokalnim merenjima. Za brzu i intuitivnu procenu stanja, podaci su kolor-kodirani: naranžasta boja označava nizak nivo, crvena boja označava opasne ili kritične vrednosti, dok bela boja označava normalne ili prihvatljive vrednosti. Ovakav prikaz omogućava korisniku da odmah uoči odstupanja i potencijalno problematična stanja u merenjima.

Kako bi se obezbedila autentičnost poslatih podataka i sprečilo neovlašćeno slanje podataka na otvoreni API, implementiran je mehanizam autentifikacije putem X-Micro-Secret-Key HTTP zaglavlja. Ovaj tajni ključ se šalje sa mikrokontrolera zajedno sa svakim zahtevom ka backendu, čime se obezbeđuje da backend prihvata podatke isključivo sa autorizovanog mikrokontrolera i onemogućava ubacivanje lažnih ili neautorizovanih podataka.

Projekat se razvija timski uz korišćenje Git verzionog sistema za kolaboraciju i arhiviranje koda. Ceo projekat je dostupan putem Git repozitorijuma na:

<https://gitea.steve-dekart.xyz/Weather-Mikro-Project>

2. Realizacija Zadatka

Tehnologije i biblioteke

Frontend aplikacije je razvijen korišćenjem React biblioteke, dok je za vizuelni izgled i UI komponente korišćen Material UI framework i TypeScript je korišćen za strožu tipizaciju i bolju preglednost koda. Ovaj pristup omogućava kreiranje modernog i preglednog korisničkog interfejsa, uključujući grafičke prikaze podataka (grafikone) koji vizualizuju temperature, vlažnost i osvetljenost.

Podaci za prikaz na grafikonu preuzimaju se sa backend endpoint-ova, gde se vrši i neophodna obrada i izračunavanje statistike. Grafikoni omogućavaju pregled istorijskih podataka u različitim vremenskim intervalima: prošlih 10 minuta, 30 minuta, 1 sat, 6 sati, 12 sati i 24 sata.

Aplikacija je implementirana kao SPA (Single Page Application), što korisnicima omogućava brzo i neprekidno pregledanje podataka bez ponovnog učitavanja stranice.

Backend aplikacije razvijen je korišćenjem Django framework-a, dok je za kreiranje REST API-ja korišćen Django REST Framework (DRF). Django omogućava definisanje endpoint-ova za mikrokontroler i frontend aplikaciju (sa automatski generisanim Swagger dokumentacijama), dok Django Admin panel pruža jednostavan način za upravljanje podacima kroz CRUD operacije za administrativne korisnike.

Django REST Framework se koristi za kreiranje API endpoint-ova, serijalizaciju modela i obradu HTTP zahteva. Za slanje obaveštenja u Telegram koristi Telegram API, koji je podešen da se izvršava periodično kao cron job na serveru.

U projektu je definisan model:

WeatherStats:

- id
- temperature
- humidityGround
- humidityAir
- light

Za produkciju je korišćena MySQL baza podataka, dok je za testiranje i razvoj korišćen SQLite.

Za projekat je korišćen **ESP32 (NodeMCU)** mikrokontroler, koji prikuplja podatke sa senzora i šalje ih na backend aplikaciju.

Za implementaciju softverskog dela na mikrokontroleru korišćene su sledeće biblioteke:

- WiFi.h – omogućava povezivanje mikrokontrolera na Wi-Fi mrežu i pristup internetu.
- HTTPClient.h – koristi se za slanje podataka ka API endpoint-ovima backend-a.
- DHT.h – biblioteka za komunikaciju sa DHT22 senzorom i očitavanje temperature i vlažnosti vazduha.

Pored navedenih, koriste se i analogni pinovi za očitavanje dodatnih senzora:

- Senzor vlažnosti zemljišta (povezan na analogni pin SOIL_PIN)
- Foto-otpornik (LDR) za merenje nivoa svetlosti (povezan na analogni pin LDR_PIN)

Struktura projekta

Backend

Backend deo projekta organizovan je na modularan način kako bi razvoj i održavanje bili jednostavni. Glavni folderi i fajlovi su:

```
|— manage_dev.py
|— manage.py
|— project
|   |— asgi.py
|   |— __init__.py
|   |— serializers.py
|   |— settings
|   |   |— base.py
|   |   |— dev.py
|   |   |— __init__.py
|   |   |— prod.py
|   |— settings_context.py
|   |— urls.py
|   |— wsgi.py
|— requirements.txt
|— weather
|   |— admin.py
|   |— apps.py
|   |— __init__.py
|   |— management
|   |   |— commands
|   |       |— send_telegram_stats.py
|   |— migrations
|   |   |— 0001_initial.py
|   |   |— 0002_rename_humidityair_weatherstats_humidity_air_and_more.py
|   |   |— __init__.py
|   |— models.py
|   |— serializers.py
|   |— tests.py
|   |— urls.py
|   |— utils.py
|   |— views.py
```

models.py – definicija glavnog modela WeatherStats koji čuva podatke sa mikrokontrolera.

serializers.py – pretvaranje modela u JSON za API endpoint-ove.

views.py / urls.py – logika i rute API-ja za mikrokontroler i frontend.

admin.py – upravljanje podacima kroz Django Admin panel.

management/commands/send_telegram_stats.py – skripta za slanje periodičnih obaveštenja na Telegram (cron job).

settings/ – konfiguracija projekta po okruženju (razvoj / produkcija).

Frontend

Frontend aplikacija je razvijena korišćenjem React biblioteke i Vite alata za izgradnju projekta, dok je TypeScript korišćen za strožu tipizaciju i bolju preglednost koda. Glavna hijerarhija foldera i fajlova je:

```
├─ eslint.config.js
├─ index.html
├─ package.json
├─ package-lock.json
├─ public
│   └─ vite.svg
├─ README.md
├─ src
│   └─ api
│       └─ getWeatherData.ts
│   └─ App.css
│   └─ App.tsx
│   └─ assets
│       └─ react.svg
│   └─ components
│       └─ HistoryTable.tsx
│       └─ LiveData.tsx
│       └─ MeteoData.tsx
│       └─ Title.tsx
│       └─ WeatherChart.tsx
│   └─ index.css
│   └─ main.tsx
│   └─ utils
│       └─ functions.ts
├─ tsconfig.app.json
├─ tsconfig.json
├─ tsconfig.node.json
└─ vite.config.ts
```

src/api/ – sadrži funkcije za preuzimanje podataka sa backend API-ja (REST endpoint-ova).

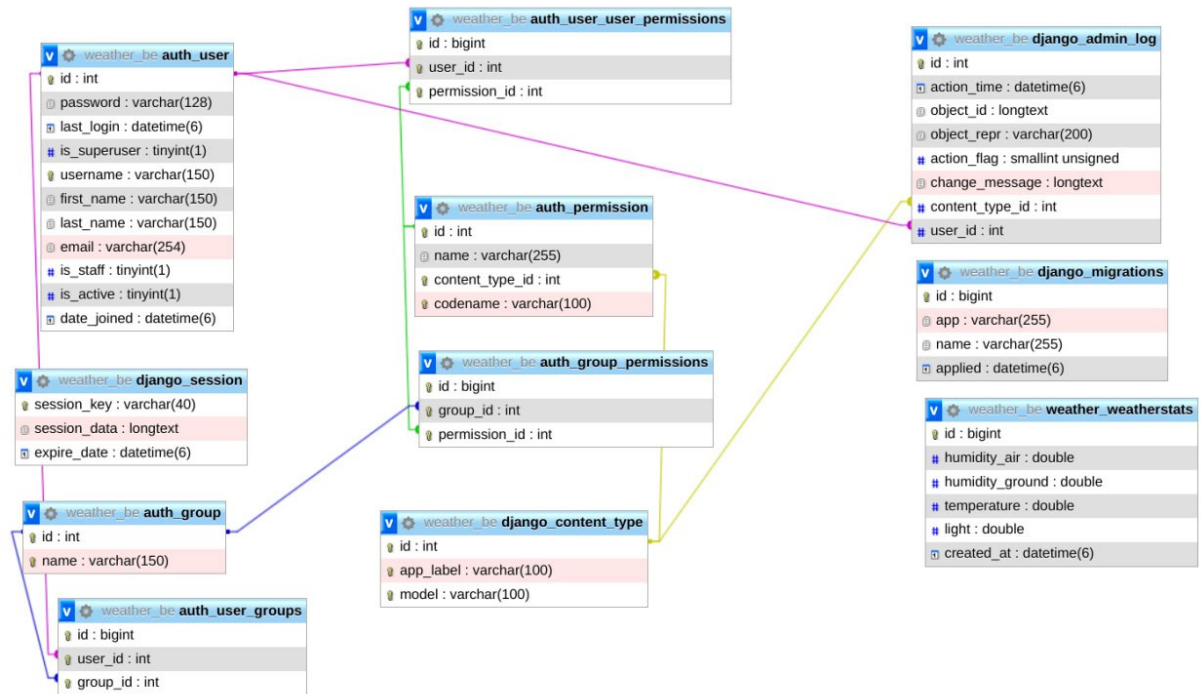
src/components/ – modularne React komponente za prikaz podataka i grafikona.

src/utils/ – pomoćne funkcije koje se koriste unutar više komponenti.

App.tsx i main.tsx – centralne tačke aplikacije koje kombinuju komponente i pokreću SPA.

Vite + TypeScript konfiguracije – omogućavaju brzu izgradnju, moderni razvojni workflow i tipizaciju koda

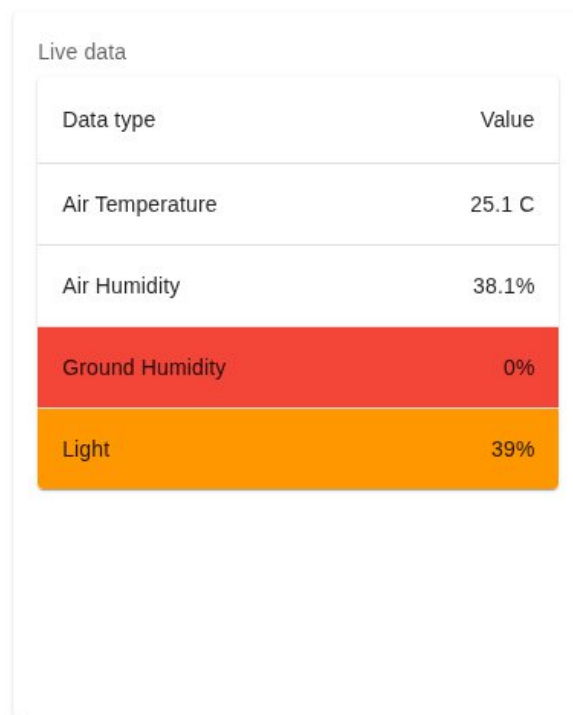
3. Struktura baze podataka



4. Opis funkcionalnosti

LiveData widget

LiveData je React komponenta koja prikazuje trenutne vrednosti senzora sa mikrokontrolera u realnom vremenu. Komponenta periodično poziva backend API da preuzme najnovije podatke i automatski osvežava tabelu sa temperaturom, vlažnošću vazduha, vlažnošću zemljišta i nivoom svetlosti. Ako podaci nisu dostupni, korisniku se prikazuje obaveštenje da mikrokontroler možda nije povezan. Tabela je napravljena pomoću Material UI biblioteke, što omogućava pregledan i moderan prikaz. Na osnovu očitanih vrednosti pozadinska boja reda se menja da obavesti korisnika o statusu.



Data type	Value
Air Temperature	25.1 C
Air Humidity	38.1%
Ground Humidity	0%
Light	39%

Slika 1. Podaci uživo

HistoryTable

HistoryTable je React komponenta koja prikazuje istorijske podatke sa mikrokontrolera u tabelarnom formatu. Komponenta učitava podatke sa backend API-ja za određenu stranicu i omogućava pregled starih očitavanja, uključujući temperaturu, vlažnost vazduha, vlažnost zemljišta, nivo svetlosti i vremenski pečat. Pagination komponenta omogućava jednostavno prebacivanje između stranica istorije, dok je tabela kreirana pomoću Material UI biblioteke za pregledan i moderan prikaz. Na osnovu očitanih vrednosti pozadinska boja ćelije se menja da obavesti korisnika o statusu.

Full sensor reading history

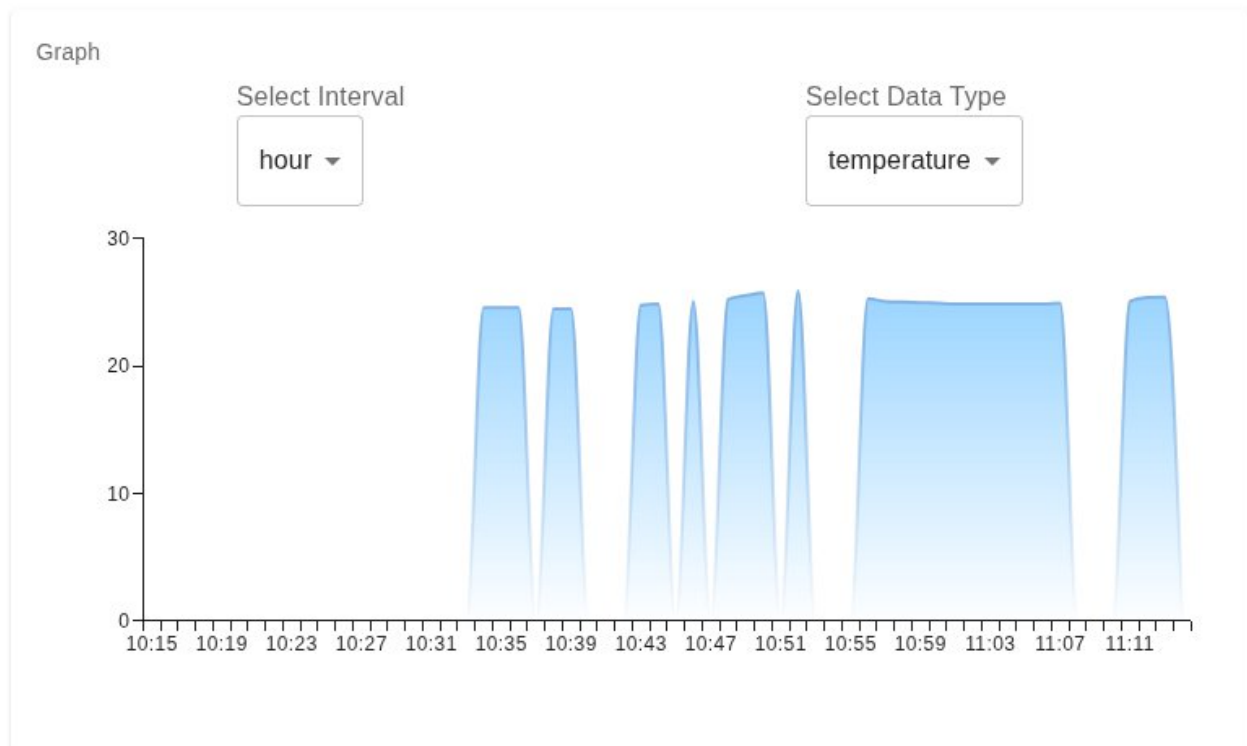
Air Temperature	Air Humidity	Soil Moisture	Light	Timestamp
25.1	36.9	0	68	16/01/2026 10:48
25.1	36	0	67	16/01/2026 10:46
24.9	37	0	67	16/01/2026 10:44
24.8	37.5	0	66	16/01/2026 10:43
24.8	40.5	0	66	16/01/2026 10:43
24.5	37.2	0	66	16/01/2026 10:39
24.5	37.4	0	67	16/01/2026 10:38
24.6	35.7	0	68	16/01/2026 10:36
24.6	40.2	0	67	16/01/2026 10:35
24.6	39	0	67	16/01/2026 10:34
22.5	26.4	0	90	09/01/2026 11:27
22.5	26.3	0	90	09/01/2026 11:27
22.5	26.3	0	90	09/01/2026 11:26
22.6	26.2	0	89	09/01/2026 11:26
22.6	26.3	0	90	09/01/2026 11:25
22.6	26.4	0	90	09/01/2026 11:24
22.6	26.2	0	89	09/01/2026 11:24
22.6	26.1	0	90	09/01/2026 11:23
22.7	26.1	0	89	09/01/2026 11:23
22.7	25.6	0	89	09/01/2026 11:22

< 1 ... 4 5 6 ... 33 >

Slika 2. Istorija očitavanja

WeatherChart

WeatherChart je React komponenta koja prikazuje istorijske podatke sa mikrokontrolera u obliku linijskog grafikona. Komponenta preuzima statistiku sa backend API-ja za različite vremenske intervale (od 10 minuta do 24 sata) i omogućava prikaz različitih tipova podataka: temperatura, vlažnost vazduha, vlažnost zemljišta i nivo svetlosti. Korisnik može odabrati interval i tip podataka, a grafikon se automatski osvežava. Vizuelni prikaz koristi Material UI X-Charts sa gradientnom bojom i preglednim formatiranjem ose X, što omogućava jasan i moderan prikaz promena u vremenskom toku.



Slika 3. Grafikon

MeteoData

MeteoData je React komponenta koja prikazuje vremenske podatke preuzete sa Open Meteo API-ja za određene koordinate. Komponenta učitava prognozu i prikazuje je u tabeli sa dnevnom i noćnom temperaturom, vlažnošću i brzinom vetra. U slučaju greške ili dok podaci ne stignu, korisniku se prikazuje odgovarajuća poruka. Tabela koristi Material UI komponente za pregledan i moderan prikaz meteo informacija.

Meteo API				
Date	Day Temp	Night Temp	Humidity	Wind Speed
16/01/2026	1.6°C	-0.6°C	84.5%	19.4 km/h
17/01/2026	-0.0°C	-7.9°C	84.2%	14.0 km/h
18/01/2026	-2.2°C	-9.5°C	87.4%	8.9 km/h
19/01/2026	-5.5°C	-10.3°C	91.4%	9.4 km/h
20/01/2026	-5.4°C	-10.6°C	91.5%	8.2 km/h
21/01/2026	-5.8°C	-12.9°C	88.9%	7.0 km/h
22/01/2026	-3.2°C	-11.3°C	85.7%	10.6 km/h
23/01/2026	-2.6°C	-9.8°C	82.3%	11.5 km/h

Slika 4. Meteo api prognoza

5. Korišćena Literatura

Web izvori

W1. https://web.archive.org/web/20240208091140/https://docs.espressif.com/projects/espressif/en/latest/esp32/_images/esp32-devkitC-v4-pinout.png - Datasheet za esp32 WROOM mikrokontroler

W2 <https://randomnerdtutorials.com/esp32-http-get-post-arduino/> – HTTP metode za esp32